50.051 Programming Language Concepts (Y2026)

Group 11:

1007687	Wonna Tun
1008139	Teng Thian Foo
1008050	Pathomphu Kanapornthada
1007998	Chadrick Liang Jing

Table of Contents

Project Report: Abyss Walker (Procedural C90 Game)

1. Project Context

Abyss Walker is an offline 2D tactical roguelike engine developed entirely in strict C90. The project moves away from standard real time rendering libraries to focus on procedural generation, state-driven gameplay, parsing and file I/O.

The game algorithmically generates a maze, drops the player into a grid based cavern. The objective is to navigate the corridors to reach an exit while being hunted by an enemy entity (the Monster), all while trying to minimize the number of steps taken. The player can find a weapon (the Bow) to eliminate the enemy using line of sight mechanics. The game combines clean C programming, persistent binary save states (.sav files), custom text & binary parsing with dynamic configuration loading, state machines, cross-platform reproducibility and a parsed leaderboard system to track the most efficient escape routes.

2. System Architecture & How It Works

The system operates on a 2D grid represented by a fixed-size character array and a turn based loop. On launch, the program reads config.txt to determine maze width, maze height, random seed, and density. It then generates a procedural maze using an iterative Depth First Search (DFS) backtracker over a cell grid. After generation, the game enters a turn-based loop in which the player enters commands, the game updates the player state, the enemy chooses its next move, and the map is redrawn in the terminal. We ensured the maze lanes were wide enough for maneuverability and tactical positioning in evading or killing the monster.

The player can use string commands to move, save, load the current game or quit. The player can also pick up a bow and fire only when there is a clear horizontal or vertical line of sight to the monster with no walls in the way. The bow uses a custom line of sight algorithm that iterates across the 2D array to ensure no wall obstructs the path between the player and the enemy before allowing a successful hit. The enemy uses Breadth First Search (BFS) pathfinding, chasing the player when close enough, otherwise patrolling toward random floor tiles.

3. Inputs, Outputs & Deliverables

3.1 Inputs

The project processes multiple input files. config.txt is a text configuration file that determines generation and gameplay parameters. leaderboard.csv is read as text data when existing leaderboard records are loaded. maze.sav is read as a binary input when a previous save is restored.

3.2 Outputs

The project also produces both text and binary outputs. leaderboard.csv is written as a text file containing player initials and step counts. maze.sav is written as a binary file containing the full serialized game state. During execution, the program renders the maze, player, enemy, weapon and status text directly to the terminal.

3.3 Submission Deliverables

The code deliverables include C source files, the game.h header, the Makefile, the README, the configuration file, and sample data files generated through play. This report explains how the constraints from the project brief are satisfied and documents the system design, development process, team contributions and lessons learned.

4. How we worked with the constraints

4.1 Clean C Programming

The entire project is implemented in C with no external libraries. The code is separated into modules for the main game loop, maze generation and rendering, enemy behavior, configuration parsing and save/load logic. Shared declarations are stored in game.h. The project compiles successfully with gcc using -Wall -Werror -ansi -pedantic, which aligns with the requirement for strict and clean C programming.

4.2 File Reading and Writing

The project demonstrates both text and binary file processing. The text side includes reading config.txt and reading and writing leaderboard.csv. The binary side includes reading and writing maze.sav. This directly satisfies the requirement that the system should process files as input and output during its operation.

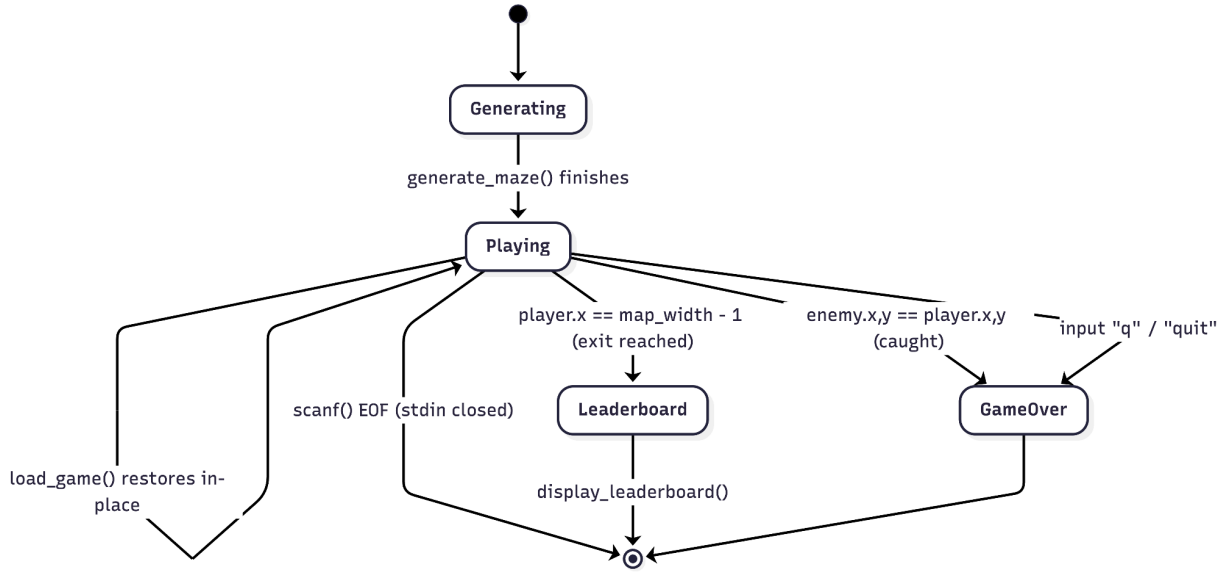
4.3 Custom Parser

The config parser in config.c reads a custom key=value format without relying on external parsing utilities. It supports the keys width, height, seed, and density. The parser trims leading and trailing whitespace, accepts spacing around the equals sign, and converts value strings into integers.

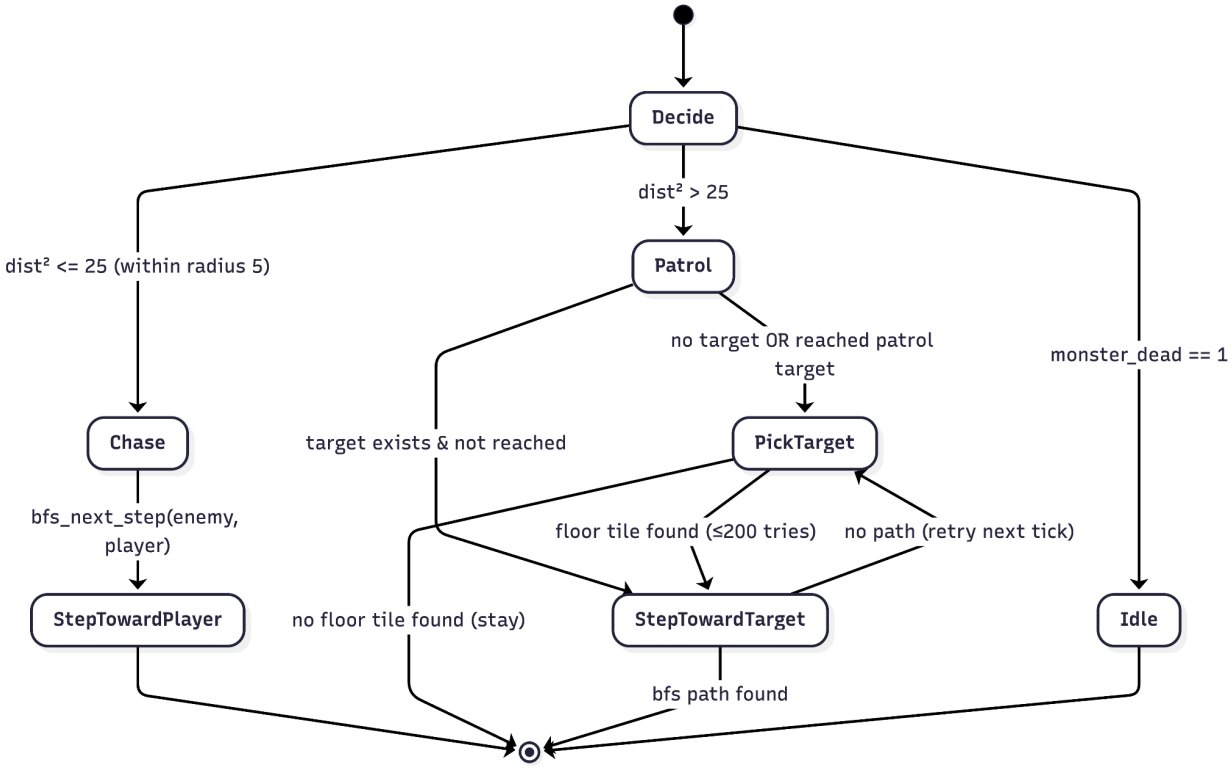
The parser satisfies the requirement that incorrect input must be detected robustly. It rejects malformed lines with missing equals signs, lines containing multiple equals signs, empty keys, empty values, duplicate keys, unknown keys, invalid integers and out-of-range values. It reports line-numbered parse errors and only commits parsed settings if the full file is valid. Otherwise, it safely falls back to default values.

4.4 State Machines

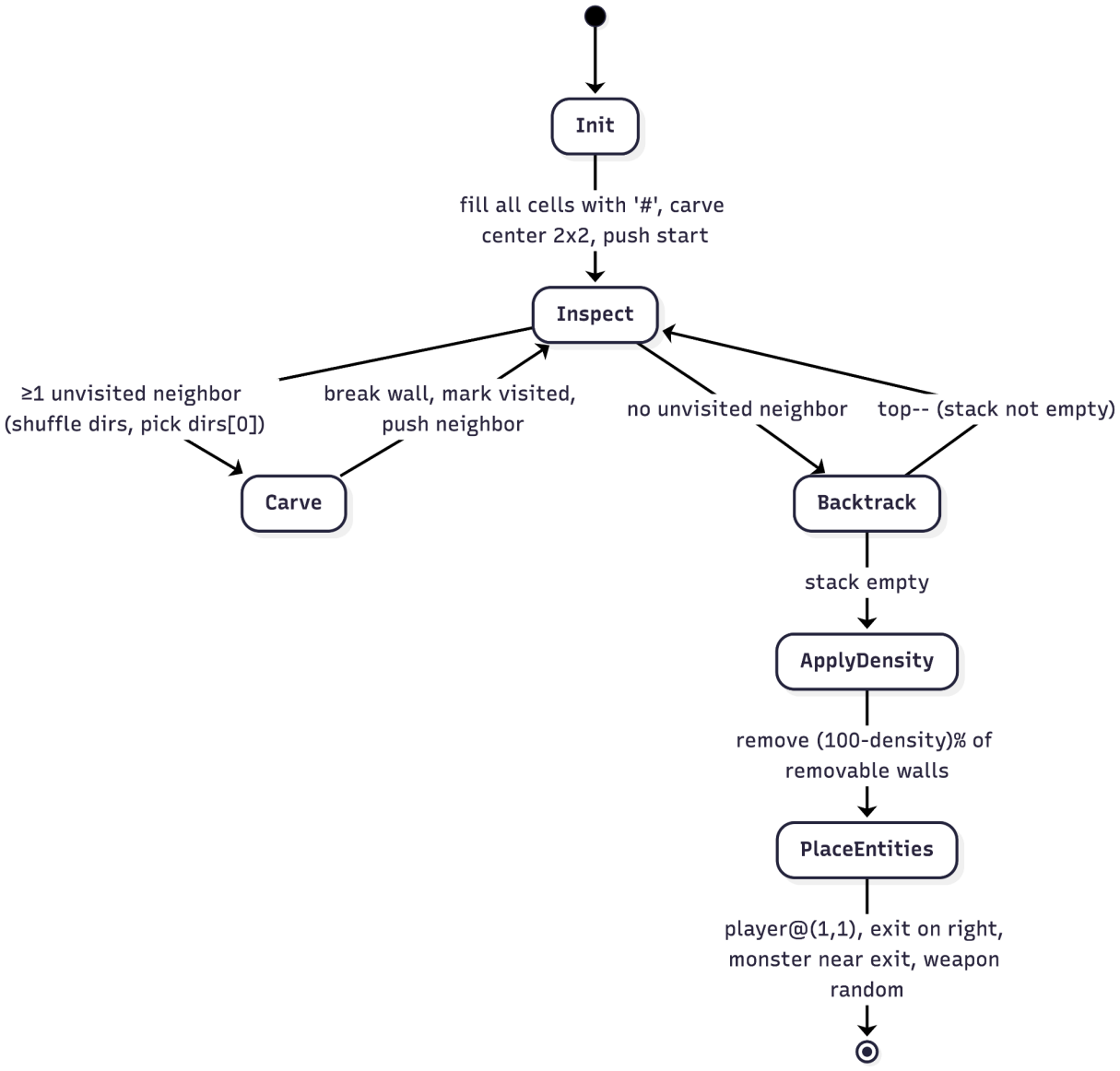
The main game flow is controlled by an explicit state machine defined by the GameState enum: STATE_GENERATING, STATE_PLAYING, STATE_GAME_OVER, and STATE_LEADERBOARD. These states are used in main.c to control generation, active play, losing conditions and score display.



A second state-like behavior exists in the enemy system. The monster alternates between chase behavior and patrol behavior depending on player proximity. Implemented as conditional logic, it demonstrates state-dependent decision-making in gameplay.

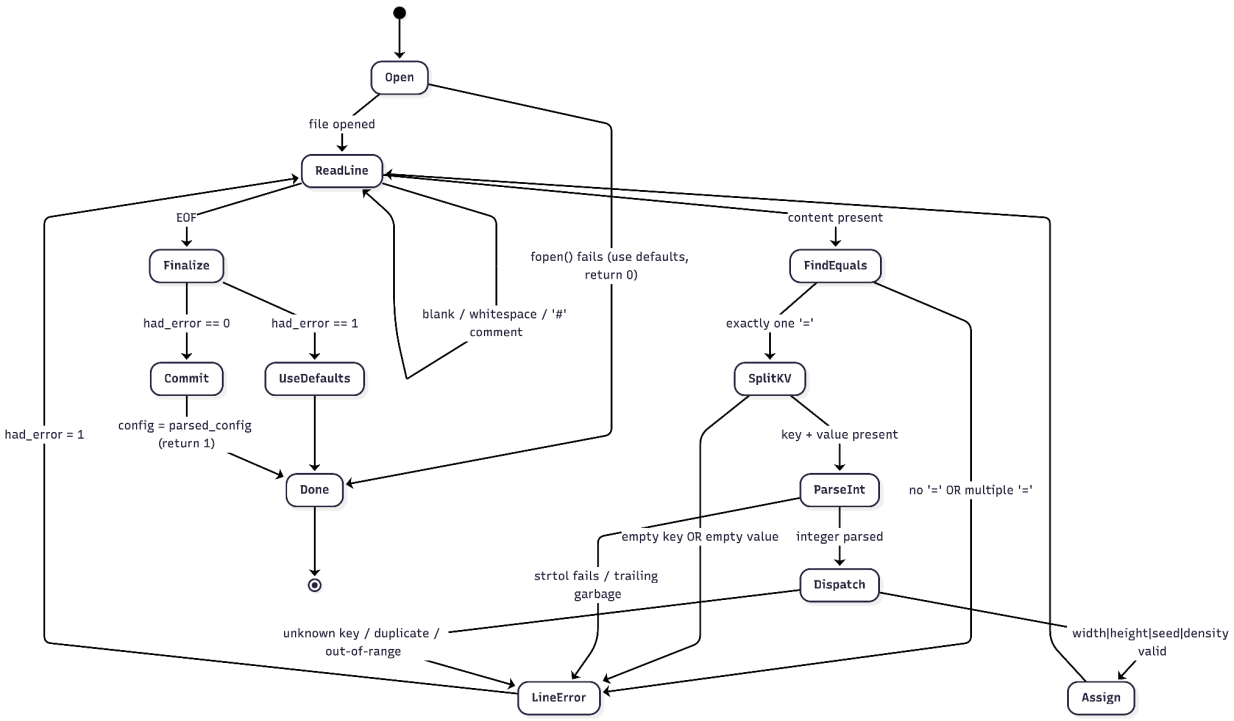


A third state machine appears in the maze generator in map.c. The iterative DFS backtracker cycles between three main states for each cell on its stack: inspect, carve and backtrack. From inspect it transitions to carve when at least one unvisited neighbor exists — breaking the shared wall, marking the neighbor visited, and pushing it onto the stack or to backtrack when no unvisited neighbors remain, in which case it pops the stack. Generation terminates when the stack is empty, after which the machine advances to a density pass that removes a proportion of internal walls and a final placement step for the player, exit, monster, and weapon. Representing the algorithm as an explicit state machine over a manual stack avoids recursion and keeps maximum stack depth bounded by the number of cells.



A fourth state machine drives the configuration parser in config.c, which processes config.txt line by line. Each line advances through a fixed sequence of states: read, classify (skipping blanks and comments), locate the = separator, split key and value, parse the value as an integer, and dispatch on the key to check for duplicates and range validity. Any of these states

may transition into a shared error state that records the offending line number and continues to the next line, so every issue in the file is reported in a single run rather than aborting on the first failure. Only at end-of-file does a terminal transition decide whether to commit the parsed values to the active configuration or discard them and fall back to defaults, satisfying the "all-or-nothing" validation requirement described in 4.3 Custom Parser.



4.5 No External Libraries

The program uses only standard C headers such as stdio.h, stdlib.h, string.h, ctype.h, limits.h, and time.h. No third-party graphics, parsing, or serialization libraries were utilized, keeping the project lightweight and compliant with the assignment constraints.

5. Detailed Implementation

5.1 Maze Generation

The maze generator in map.c uses a depth-first-search recursive backtracker over a cell grid. Each logical cell expands into a larger tile region in the display map, producing wider corridors for movement and combat. After the core maze is generated, an optional density adjustment removes some internal walls to create shortcuts and more open layouts. This allows the generated map to range from a perfect maze, with one single perfect path from start to end, to a more open cave-like structure.

5.2 Game Loop and Rendering

The main loop reads a string command, interprets it as movement or an action, updates the player position if the target tile is open, performs save/load operations when requested, and checks victory or defeat conditions. Rendering redraws the terminal display every turn by printing the current map and overlaying the player, enemy, and weapon symbols.

5.3 Enemy Behaviour

The enemy system in enemy.c uses breadth-first search to compute the first step of a shortest path toward either the player or a patrol target. If the player is within a fixed chase radius, the monster pursues directly. Otherwise, it patrols toward a randomly chosen reachable floor tile. This improvement solved an earlier problem in which simpler horizontal and vertical tracking would get stuck behind walls.

5.4 Line of Sight and Combat

The bow mechanic checks whether the player and monster are aligned on the same row or column by iterating across the 2D array and checking whether every tile between them is free of walls. Only then can the player shoot successfully.

5.5 Save and Load System

The save/load system stores the full game state, including map dimensions, cave map contents, entity locations, inventory flags, monster state, and step count.

Earlier versions of the project used direct raw-memory dumping for integers and structs. This was replaced with an explicit portable binary format.

The current implementation writes integer values as explicit 32-bit little-endian byte sequences and writes entity fields individually (`x`, `y`, `symbol`) rather than relying on compiler struct layout. This makes `maze.sav` portable across platforms such as Windows and Linux.

On load, the game validates the file signature, dimensions, flags, reconstructed entities, and loaded map contents (within the active loaded map bounds) before accepting the restored state. The loader also applies data transactionally: it first reads into temporary buffers and only commits to live game globals after all checks pass. This prevents partially loaded or corrupted data from mutating in-memory game state.

5.6 Leaderboard System

The leaderboard uses leaderboard.csv as a text file. Each record stores a three-letter set of initials and the step count. Existing records are parsed with string-processing functions, a new score is inserted, qsort orders the entries in ascending order of steps, and the results are written back to the CSV file before being displayed.

5.7 Dockerization

Dockerization was used to make Abyss Walker portable and reproducible across different environments, but it is optional rather than required. The game can still be built and run natively with gcc and make, while Docker provides an alternative execution path for users who prefer an isolated, consistent setup. The project uses a multi-stage container design, compiling in a GCC build image and running in a lightweight runtime image to reduce host dependency issues. With Docker Compose, setup is standardized (TTY, stdin, and service configuration), and persistent data is handled through mounted volumes so save files and leaderboard data can survive container restarts. This optional container workflow improves portability and reduces environment-related friction without changing the core native build process.

6. Development Process

We adopted a modular, iterative development approach. We started with a basic text parser and a single 2D array map representation, then expanded into maze generation, the game loop, the enemy behaviour algorithm, binary parser logic, leaderboard handling and finally the save/load support.

To ensure cross platform compatibility and because standard C buffers input differently across operating systems, we strictly avoided OS specific libraries and rely on a GNU-style Makefile for builds. The README documents Linux usage with make and Windows usage with either make under MSYS2 or mingw32-make under standalone MinGW-w64. The Makefile selects the appropriate executable name for Windows or non-Windows builds.

Cross-platform work also affects the save system. By replacing raw struct dumping with explicit serialization, the save file format is no longer dependent on machine endianness, integer layout, or struct padding.

By standardizing our build process with a Makefile, every member could pull the code, run make and test the executable identically on Windows and Linux. Using string-based input with scanf("%s") also simplified command parsing and avoided some input-buffer inconsistencies that can appear with character-by-character input across different terminal environments.

7. Challenges Encountered

One challenge involved enemy behavior after combat. Earlier versions could leave logic active after the monster had been removed visually, producing inconsistent behavior. This was fixed by ensuring monster rendering and movement respect the monster_dead state consistently.

Another challenge involved movement logic for the monster. Simpler directional tracking often got stuck behind maze walls. This was solved by using breadth-first search for chasing and patrolling, which made the enemy more reliable and more interesting.

A further challenge involved configuration and save-file correctness. A lenient parser could silently accept bad input and raw binary serialization could behave differently across platforms. These issues were solved through stronger parsing checks and explicit portable serialization.

Finally, modular refactoring introduced the usual C challenge of coordinating shared globals and function declarations across files. This was resolved by organizing declarations carefully in the shared header and keeping responsibilities separated by module.

8. Division of Labor

To avoid merge conflicts and ensure equal participation, the architecture was divided into four distinct components:

Wonna: Handled the refactoring of the codebase into separate scripts, built the Makefile, implemented the string based input system

Chadrick: Wrote the DFS Recursive Backtracker algorithm, managed the 2D array memory and programmed the enemy behaviour.

Thian Foo: Main game loop, managed the terminal rendering logic and handled the saving and loading of game using binary serialization.

Richy: Developed the text parser for config.txt and leaderboard.csv, implemented the leaderboard sorting.

9. Key Learnings

This project reinforced several important programming language concepts. Parsing text files, writing raw structs and arrays to a binary file helped with our understanding of contiguous memory blocks and how C views data.

We also learned that state persistence, procedural generation and nontrivial game logic can be achieved with foundational C features such as arrays, loops, structs and file I/O alongside game algorithms, without depending on large external engines.

The work on save-file portability also highlighted why file formats should be designed explicitly instead of writing compiler-dependent in-memory layouts directly to disk.

Lastly, refactoring the project taught us the importance of decoupling code. Keeping the file I/O separate from the game loop made debugging significantly easier and allowed our team to work in parallel without overwriting each other's work.

11. Build and Run Instructions

Requirements: gcc and GNU make.
On Linux, build with make and run with ./abyss_walker.

On Windows, build with make in MSYS2 or mingw32-make in standalone MinGW-w64 and run with .\abyss_walker.exe.
The repository README also documents configuration options, controls, cleaning commands, and cross-platform notes.

12. Link to Github

https://github.com/RichyYhcir/PLC

13. Conclusion

Abyss Walker satisfies all the major project requirements. It is a complete C project with a clear gameplay purpose, robust text parsing, both text and binary file processing, explicit state-machine-driven game flow and a cross-platform build and save strategy. The project also demonstrates attention to delivery through documentation, modular structure and reproducible compilation. Overall, it combines the core ideas taught in the course into a single cohesive and playable system.